1 Teori I

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

Ett use case beskriver hur aktörer (Actors) använder systemet för att uppnå ett visst mål.

- ○ Falskt
- ○ Sant ✔

Ett use case diagram visar hur man använder en viss klass

- ○ Falskt ✔
- ○ Sant

En domänmodell är ett slags karta över alla use cases i domänen, och hur de hänger samman

- ○ Falskt ✔
- ○ Sant

Design patterns beskriver hur man bäst strukturerar användargränssnitt.

- ○ Falskt ✔
- ○ Sant

"Unit Testing" är ett speciellt testramverk för spelmotorn "Unity".

- ○ Sant
- ○ Falskt ✔

Ett Use Case kan användas som ett testfall för ett system.

- ○ Sant ✔
- ○ Falskt

Ett Design Pattern är ett generellt förslag på lösning till ett vanligt förekommande programvarudesignproblem

- ○ Sant ✔
- ○ Falskt

Totalpoäng: 7

2 Teori II

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

Ett klassdiagram visar alla objekt som skapas av varje klass

- ○ Sant
- ○ Falskt ✔

Ett klassdiagram visar alla attribut som varje klass har, men inte vilka värden varje attribut har.

- ○ Falskt
- ○ Sant ✔

Ett klassdiagram beskriver hur klasser och objekt samarbetar.

- ○ Falskt ✔
- ○ Sant

Metoder i en klass kan vara public, protected, eller private.

- ○ Sant ✔
- ○ Falskt

En metod som är deklarerad som public får inte använda sig av attribut (i samma klass) som är private

- ○ Sant
- ○ Falskt ✔

Ett sekvensdiagram beskriver hur klasser anropar andra klasser i en sekvens med händelser

- ○ Falskt ✔
- ○ Sant

I ett sekvensdiagram kan man se vilka klasser som objekten är instanser av.

- ○ Sant ✔
- ○ Falskt

Interaktionsdiagram visar vilka metodanrop objekt gör på andra objekt.

- Sant ✔
- Falskt

Man gör ett interaktionsdiagram för varje systemhändelse.

- Falskt
- Sant ✔

Totalpoäng: 9

3 GRASP Patterns

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

Det blir en bättre design om man delar på klasser så att man har några som bara innehåller data, och andra som abra innehåller metoder.

- ○ Falskt ✔
- ○ Sant

En Information Expert är ansvarig för att hämta information från användargränssnittet.

- ○ Sant
- ○ Falskt ✔

En Controller använder man för att gömma ett mer komplicerat arbetsflöde bakom en klass med ett enklare gränssnitt.

- ○ Falskt
- ○ Sant ✔

En Controller förekommer bara i ekonomi-program.

- ○ Falskt ✔
- ○ Sant

Enligt High Cohesion skall man se till att varje klass skall ha så få olika ansvarsområden som möjligt.

- ○ Sant ✔
- ○ Falskt

Pure Fabrication handlar om att man har en klass som skapar instanser av andra abstrakta klasser.

- ○ Falskt ✔
- ○ Sant

Ett sätt att sträva efter Low Coupling är genom att använda ett interface snarare än en konkret klass

- ○ Sant ✔
- ○ Falskt

Totalpoäng: 7

4 Design Patterns

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

Strategy pattern använder sig av polymorfism

- ○ Sant ✔
- ○ Falskt

Abstract Factory används för att skapa rätt typ av objekt givet ett visst kontext, där resten av systemet inte behöver veta exakt vilken typ objektet är.

- ○ Falskt
- ○ Sant ✔

En Observable är en klass med data som andra klasser kan vara intresserade av

- ○ Sant ✔
- ○ Falskt

Strategy pattern går ut på att man kapslar in och gömmer undan det beteendet som kan förändras, och presenterar ett stabilt gränssnitt mot resten av systemet

- ○ Sant ✔
- ○ Falskt

I Strategy pattern är det klassen med rollen «Abstract Strategy» som resten av systemet använder sig av.

- ○ Falskt ✔
- ○ Sant

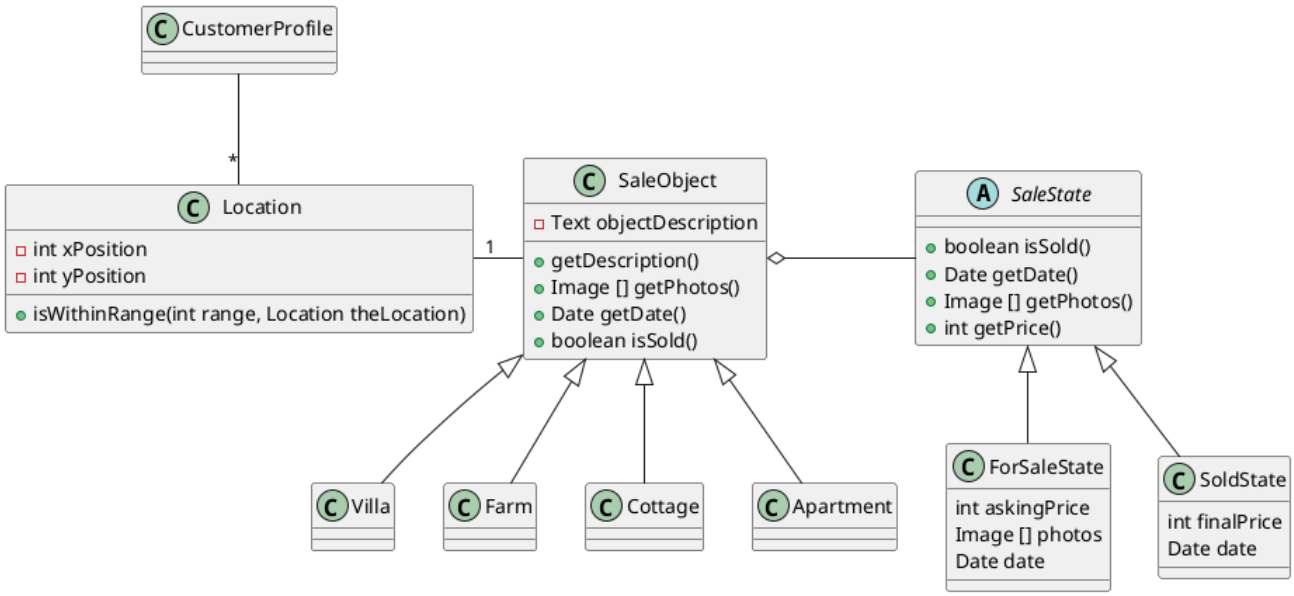
Man använder Strategy pattern när ett system är stateless för då kan man inte använda State pattern.

- ○ Sant
- ○ Falskt ✔

Totalpoäng: 6

5 Klassdiagram

Betrakta nedanstående klassdiagram:



Klassdiagrammet beskriver en marknadsplats för hus (tänk Hemnet) där man kan se vilka hus som finns till salu på en karta. I sin användarprofil anger man en (eller flera) Location, och sedan kan man söka efter hus i en viss radie runt den punkten (t.ex. Ge mig alla hus inom en radie på 50 km från Karlskrona Centrum). Man kan också se vilka hus som har sålts i samma region, och vad de kostade.

För varje påstående nedan, markera om diagrammet stödjer påståendet (sant) eller inte stödjer påståendet (falskt).
(+1 för rätt svar, ingen förändring för fel svar)

När man skapar ett object av typen Location så kommer det automatiskt skapas ett objekt av typen SaleObject.

- ○ Falskt ✔
- ○ Sant

Det finns inga metoder för att ändra på värdena i ett Location-objekt. Har man väl skapat en :Location så går den med andra ord inte att flytta.

- ○ Sant ✔
- ○ Falskt

SaleObject, SaleState, ForSaleState, och SoldState ingår tillsammans i designmönstret State pattern.

- ○ Falskt
- ○ Sant ✔

klasserna Villa, Farm, Cottage, och Apartment använder sig av Polymorfism för att hantera de skilnader som finns mellan de olika typerna av hus.

- ○ Falskt
- ○ Sant ✔

f:Farm består av flera åkrar som var och en har en referens till ett objekt av typen Location.

- Sant
- Falskt ✔

a:Farm, b:Cottage, och c:Apartment har alla tre en referens till samma :Location - objekt.

- Falskt
- Sant ✔

I metoden SaleObject::isSold() finns kodraden myState = new SaleState(); .

- Sant
- Falskt ✔

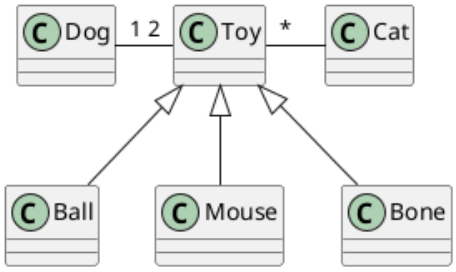
CustomerProfile har referenser till två olika :Location-objekt som har samma värden på xPosition och yPosition.

- Falskt
- Sant ✔

Totalpoäng: 8

6 Relationer mellan Klasser

Betrakta följande klassdiagram:



Notera att detta diagram fokuserar på relationerna mellan klasserna, och går inte in på detaljer i vilka metoder eller attribut som finns.

För varje påstående nedan, markera om diagrammet stödjer påståendet (sant) eller inte stödjer påståendet (falskt). (+1 för rätt svar, ingen förändring för fel svar)

tofflan:Cat äger b1:Ball

- Falskt
- Sant ✔

lasse:Dog äger b2:Ball och bone1:Bone

- Falskt
- Sant ✔

lasse:Dog och ludde:Dog tuggar på varsin ända av bone1:Bone

- Sant
- Falskt ✔

lasse:Dog får en ny leksak b3:Ball i julklapp. Då måste han glömma bort antingen b2:Ball eller bone1:Bone

- Falskt
- Sant ✔

odd:Cat har en m1:Mouse och en bird:Toy

- Falskt
- Sant ✔

Syskonen lau:Cat och leo:Cat leker med samma m2:Mouse

- Sant ✔
- Falskt

tofflan:Cat har en std::vector (Eller en java.util.Vector), där de första 42 objekten är av typen Ball

- Falskt
- Sant ✔

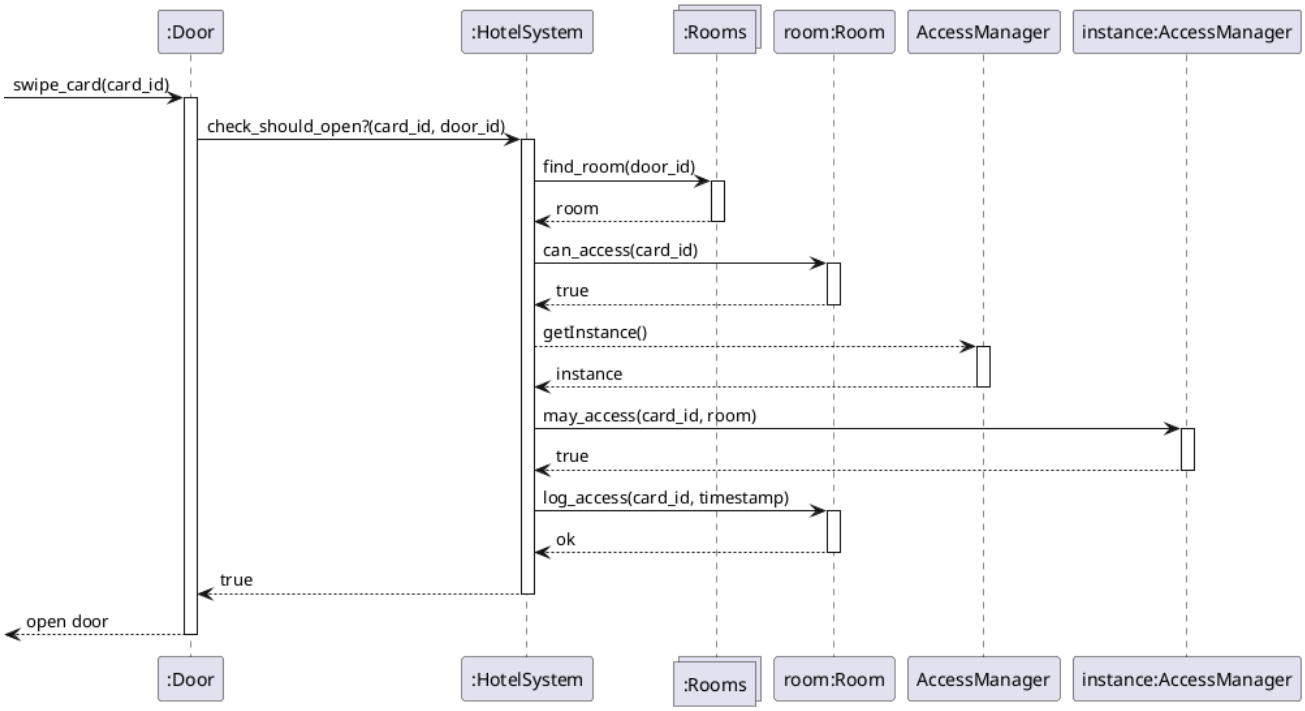
lasse:Dog och tofflan:Cat leker med b4:Ball tillsammans.

- Falskt
- Sant ✔

Totalpoäng: 8

7 Interaktionsdiagram

Betrakta nedanstående sekvensdiagram:



Systemet beskriver en del av ett hotellsystem och vad som händer när man försöker öppna en dörr med ett nyckel-kort.

För varje påstående nedan, markera om diagrammet stödjer påståendet (sant) eller inte stödjer påståendet (falskt).
(+1 för rätt svar, ingen förändring för fel svar)

:Door är en Controller

- ○ Sant
- ○ Falskt ✔

:HotelSystem är en Controller

- ○ Falskt
- ○ Sant ✔

:Rooms är en samling

- ○ Falskt
- ○ Sant ✔

Objectet room:Room måste ligga som ett attribut på klassen :HotelSystem .

- ○ Sant
- ○ Falskt ✔

Klassen AccessManager är en Singleton

- ○ Sant ✔
- ○ Falskt

:AccessManager är «context» klass i ett Strategy Pattern

- ○ Falskt ✔
- ○ Sant

Det hade hjälpt low coupling om metoden Room::can_access(card_id) hade direkt kollat med :AccessManager.

- ○ Falskt
- ○ Sant ✔

metoden find_room() ligger i klassen HotelSystem

- ○ Falskt ✔
- ○ Sant

klassen Room måste minst ha metoderna find_room(), can_access(), och log_access()

- ○ Sant
- ○ Falskt ✔

Enligt High Cohesion borde :Rooms själv bestämma om dörren skall öppnas eller inte.

- ○ Sant
- ○ Falskt ✔

Totalpoäng: 10

Betygsgränser

Betygsgränserna för den här tentan är:

Betyg	Procent	Poäng
MAX	100%	55
A	90%	50
B	80%	44
C	70%	39
D	65%	36
E	60%	33

Lycka till!